Integration project

1. Description

The objective of the project is to develop a data mining tool to load, process and analyze information from multimedia content creators hosted on different platforms. To this end, an application will be developed with a three microservices architecture, shown in Figure 1. The *PeerTubeMiner* and *DailymotionMiner* services will aim to read data from PeerTube and YouTube, respectively, and send it to *VideoMiner* for storage using a common data model. *VideoMiner*, on the other hand, will store the data and offer it to the outside world via a REST API so that other applications can query and analyze it.

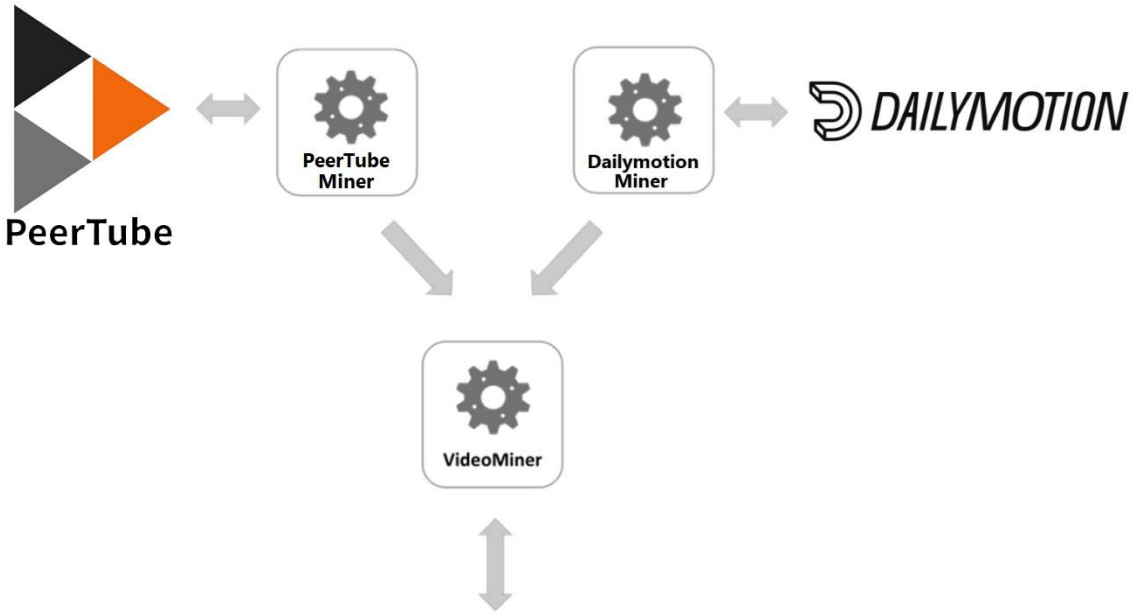

Figure 1. General view of the system architecture

In what follows, we detail the operation of each service:

PeerTubeMiner. This is an adapter service that will read data from the PeerTube REST API and send it to VideoMiner using the appropriate data model (Figure 2). In a real data engineering scenario this process could be performed periodically (e.g. every 24 hours). In this case, the service will implement a RESTful service with an operation to get the data from a channel in PeerTube and send it to VideoMiner each time it is invoked. Specifically, the operation shall be able to be invoked as follows:

POST *apipath*/{id}[?maxVideos=10&maxComments=2]

Optional parameters:

- **maxVideos**: The operation will return the number of videos per channel provided as a parameter. Default value: 10.

- **maxComments**: The operation will return the number of comments per video provided as a parameter. Default value: 2.

It is recommended to also implement an equivalent read-only operation for testing purposes, i.e. displaying the search results without sending them to VideoMiner.

DailymotionMiner. Its operation will be analogous to the previous service, but in this case the information will be taken from Dailymotion, through its REST API.

POST *apipath*/{id}[?maxVideos=10&maxPages=2]

Optional parameters:

- **maxVideos**: The operation will return yhe number of videos per channel provided as a parameter. Default value: 10.
- **maxPages**: Maximum number of result pages to return. Default value: 2.

VideoMiner. It will be the service responsible for integrating and storing all the data in an H2 database using the data model shown in Figure 2. To do so, it will implement a REST API that will allow manipulating the following resources:

- Channel. It shall implement various read and write operations to add and list channels. Among others, it will have to implement the appropriate POST operation to allow adapters to add new channel data.
- Videos. It shall implement, at a minimum, several read operations to list all videos, and search for videos by id.
- Captions. It shall implement, at a minimum, several read operations to list all captions, search by id, and return the captions for a video given its id.
- Comments. It shall implement, at a minimum, several read operations to list all comments, search for them by id, and return the comments for a video given its id.

As support material, we provide:

1. Template Project with the classes of the VideoMiner data model, necessary for the creation of the H2 database and the manipulation of the data received from the adapter services
2. Postman Test suite to validate the basic operation of VideoMiner.
3. Demo video illustrating the expected behavior of the application and how to test it.

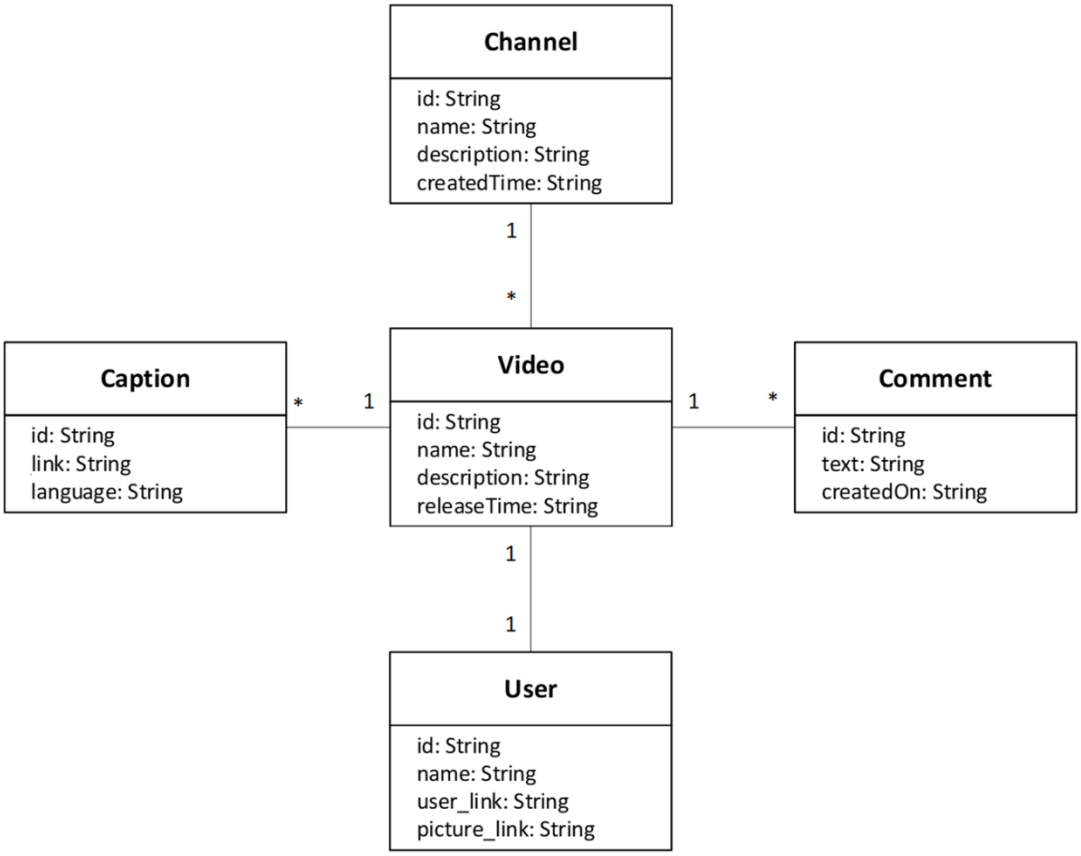

Figure 2. Data model

Important considerations:

- Captions are called “Subtitle” in the Dailymotion API.
- The user is called ‘Account’ on PeerTube and ‘Owner’ on Dailymotion.
- There are no comments in the Dailymotion API; instead, use the video tags.
- The implemented application should follow REST best practices. For example, if we ask any of the three services to return a resource with an id that does not exist, the service should return a 404 response code.

2. Evaluation Criteria

2.1. Minimum Criteria (REQUIRED FOR PASSING)

All three services must function correctly according to the instructions provided. It is imperative that all Postman tests provided as part of the supporting material pass successfully. Applications that do not conform to the architecture described above will not be accepted

2.2. Standard criteria (required to obtain a "B" grade)

In addition to the minimum criteria, the design aspects of the application will be evaluated, as well as the level of detail of the tests (unit tests with Junit and system tests with Postman) and the interactive documentation of the API (OAS). The level of participation and the clarity of the presentation during the defense will also be taken into account.

2.3. Advanced criteria (required to obtain a "A" grade)

In addition to the standard criteria, those groups that want to qualify for an "A" must implement one or more improvements agreed upon with their tutor. Some examples:

- Extension of the data model.
- Extension of the API provided by VideoMiner (e.g. pagination, filters, sorting).
- Extracting data from more platforms (e.g. Twitch).
- Use of GraphQL APIs.
- Use of authentication through API key in VideoMiner.
- Deploy all three services in the cloud.